

*"Freeze the plan before you freeze the code -
- Angular, REST, Kafka, and a real OpenShift
target everyone can defend."*

MODULE 48

Capstone Planning and Architecture

Turn the Northstar CRM brief into an Angular, REST, Kafka, PostgreSQL, and OpenShift architecture and delivery plan: stack decisions, CI/CD design, IaC strategy, an AI usage and governance plan, and an environment strategy Labs 49-51 and Module 51 build against.







MODULE 48 OVERVIEW

Freeze the Angular, REST, PostgreSQL, Kafka, and OpenShift architecture and the GitHub Actions/Terraform/Ansible delivery plan Labs 49-51 and Module 51 build against.

Week 6 opens with the Northstar CRM capstone: an Angular front end, a Spring Boot/Kafka backend, PostgreSQL, and a real OpenShift target. The team freezes the stack, CI/CD, and IaC before any code is written.

DURATION & LAB



4 Hours Theory

Stack decisions, Angular/REST/PostgreSQL/Kafka architecture, GitHub Actions and IaC design, OpenShift deployment planning, and AI usage/governance.



2 Hours Lab

Draft the architecture docs, the GitHub Actions pipeline design, the Terraform/Ansible plan, and an OpenShift environment strategy this build follows.



Scenario

Plan the Angular + OpenShift capstone for Amina (CUS-1001) and Ravi (CUS-1002) as the fixed demo fixtures.

MODULE ARC



Stack Frozen First

Angular, REST, PostgreSQL, Kafka, and OpenShift named once, here, before Labs 49-51 write a line of code.



Pipeline Designed, Not Improvised

GitHub Actions, Terraform, and Ansible planned end to end before Module 51 executes them stage by stage.



AI With a Human Gate

Every AI-assisted decision this capstone uses gets a bounded prompt and a reviewed, dated record.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours lab = 6 hours. Every decision made here -- stack, CI/CD, IaC, and AI usage -- is what Labs 49-51 and Module 51 build against, unchanged.

CAPSTONE TECHNOLOGY STACK

The Northstar CRM stack is fixed for Week 6 -- naming it precisely now avoids drift across Labs 49-51.

Layer	This Course's Technology
Frontend	Angular (TypeScript, Signals/RxJS, HttpClient) -- not React.
Backend	Spring Boot REST API, JWT/RBAC resource server.
Database	PostgreSQL via Spring Data JPA -- not Oracle.
Deployment	Docker images on Kubernetes/OpenShift -- OpenShift is the capstone target.

STACK HABITS

**Named Once**

Every technology here should show up unchanged in the relevant lab's ADRs and code.

**OpenShift For Capstone**

Module 42 teaches Kubernetes on a k3s training cluster; Labs 49-51 deploy to OpenShift.

**GitHub Actions Only**

CI/CD runs on GitHub Actions -- Bitbucket Pipelines is not used in this course.

**PostgreSQL Only**

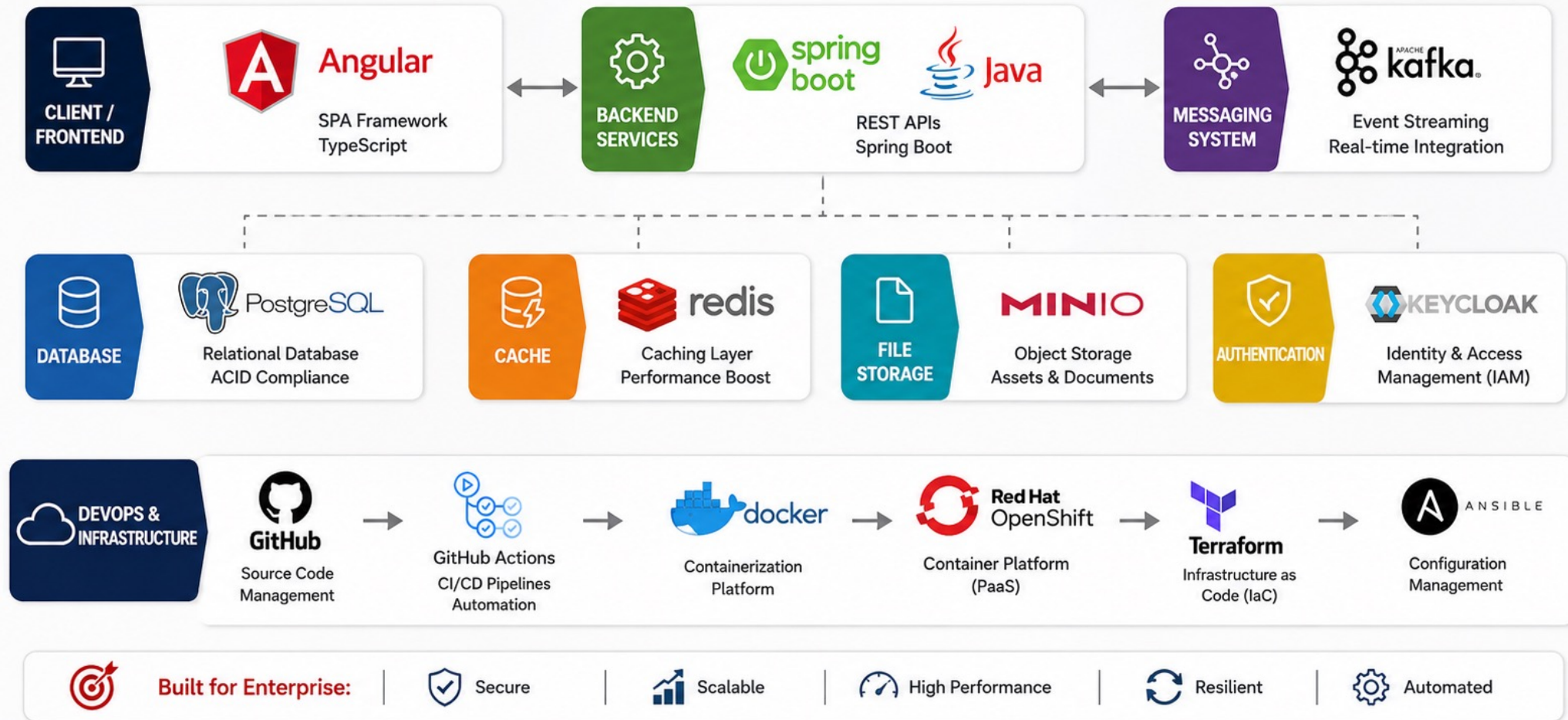
Every environment, from local to OpenShift, runs PostgreSQL - never Oracle.

KEY TAKEAWAY Angular, Spring Boot, PostgreSQL, Kafka, and OpenShift are the fixed stack -- any deviation needs its own ADR, not a silent substitution.



CAPSTONE TECHNOLOGY STACK

A modern, cloud-native, event-driven architecture for the Capstone project covering frontend, backend, data, messaging, DevOps and infrastructure.



ANGULAR FRONTEND ARCHITECTURE

Standalone components, a typed service layer, and Signals-driven state -- the Angular shape Module 50 will build.

UI STRUCTURE

Feature-based folders: customers/, interactions/, shared/.

Standalone components -- no NgModules for new feature work.

Routing lazily loads each feature via loadComponent.

Reusable presentational components stay free of HTTP calls.

FRONTEND RESPONSIBILITIES

- A typed CustomerService wraps every HttpClient call.
- Signals hold component state; RxJS handles async streams.
- Reactive forms validate before a request is ever sent.
- HTTP interceptors attach the JWT and X-Correlation-ID header.

Service shape (sketch)

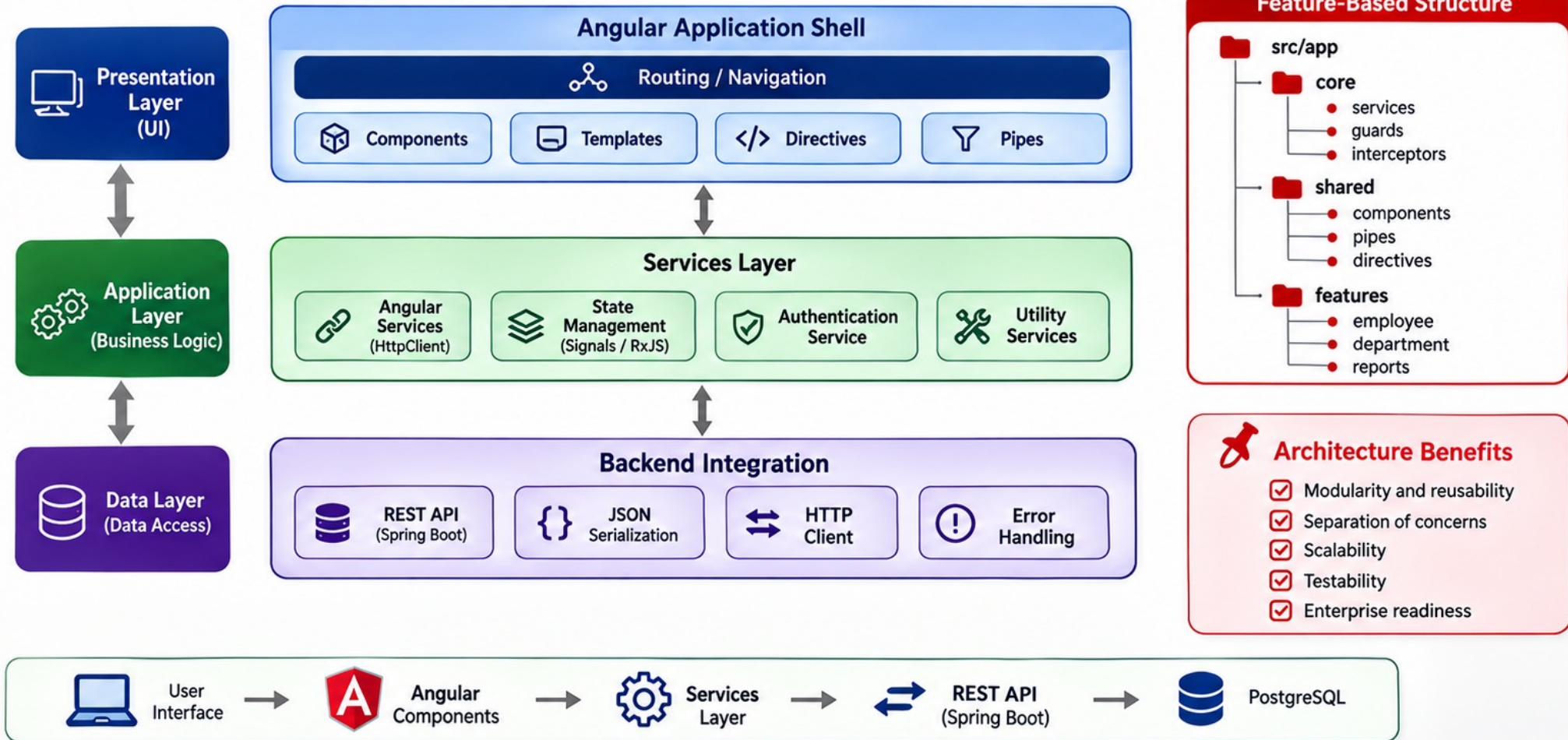
```
@Injectable({providedIn: 'root'}) export class InteractionService {  
  record(id: string, dto: CreateInteractionRequest): Observable<Interaction> { ... } }
```

KEY TAKEAWAY Angular owns presentation and validation only -- every business rule still lives in the Spring Boot service layer, never in a component.



ANGULAR FRONTEND ARCHITECTURE

Modular, scalable and maintainable architecture for enterprise Angular applications.



REST API ARCHITECTURE

A versioned, resource-oriented REST contract with Problem Details errors -- the Angular client's only view of the backend.

ENDPOINT DESIGN

Resource-oriented URIs: /api/customers/{id}/interactions.

HTTP verbs map to intent -- POST creates, PATCH updates status.

201 + Location on create; 200 on read; 204 on delete.

Pagination and sorting on every list endpoint from day one.

CONTRACT DISCIPLINE

- DTOs at the boundary -- a JPA entity never reaches the client.
- Problem Details (RFC 9457) for every 4xx/5xx response body.
- Additive fields are safe; breaking changes bump the version.
- Every request carries header X-Correlation-ID: lab-request-001.

Endpoint sketch

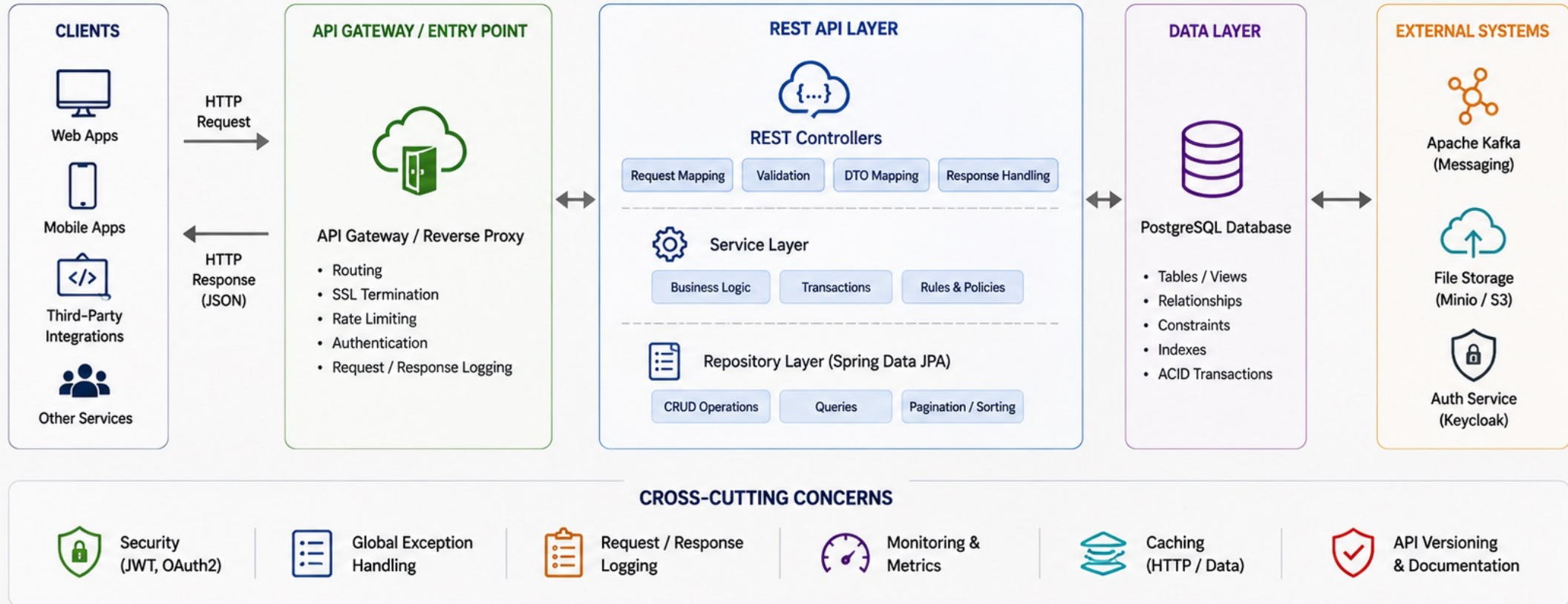
```
POST /api/customers/{id}/interactions -> 201 Created + Location
GET  /api/customers/{id}/interactions?page=0&size=20 -> 200 OK
```

KEY TAKEAWAY Unversioned, entity-leaking endpoints break Module 50's Angular client the moment the backend changes shape -- design the contract here.



REST API ARCHITECTURE

A stateless, client-server architecture that uses HTTP and REST principles to provide scalable, secure and interoperable services.



Principles: Stateless • Client-Server • Cacheable • Layered System • Uniform Interface



Scalable. Secure. Maintainable. Enterprise Ready.

POSTGRESQL PERSISTENCE ARCHITECTURE

PostgreSQL is the system of record for every environment in this course -- schema owned by Flyway, never by Hibernate auto-DDL.

SCHEMA & MIGRATION

Flyway numbered migrations own the schema -- ddl-auto stays validate.

Customer and Interaction tables, keyed by UUID primary keys.

An index on (customer_id, occurred_at) for timeline lookups.

Migrations run identically local, CI, and OpenShift.

DATA ACCESS RULES

- Spring Data JPA repositories -- no raw JDBC in the service layer.
- Transactions wrap persist-then-publish inside one boundary.
- CI runs a real PostgreSQL (Testcontainers), not just H2.
- Never Oracle, in any lab, demo, or documentation.

Migration naming (Flyway)

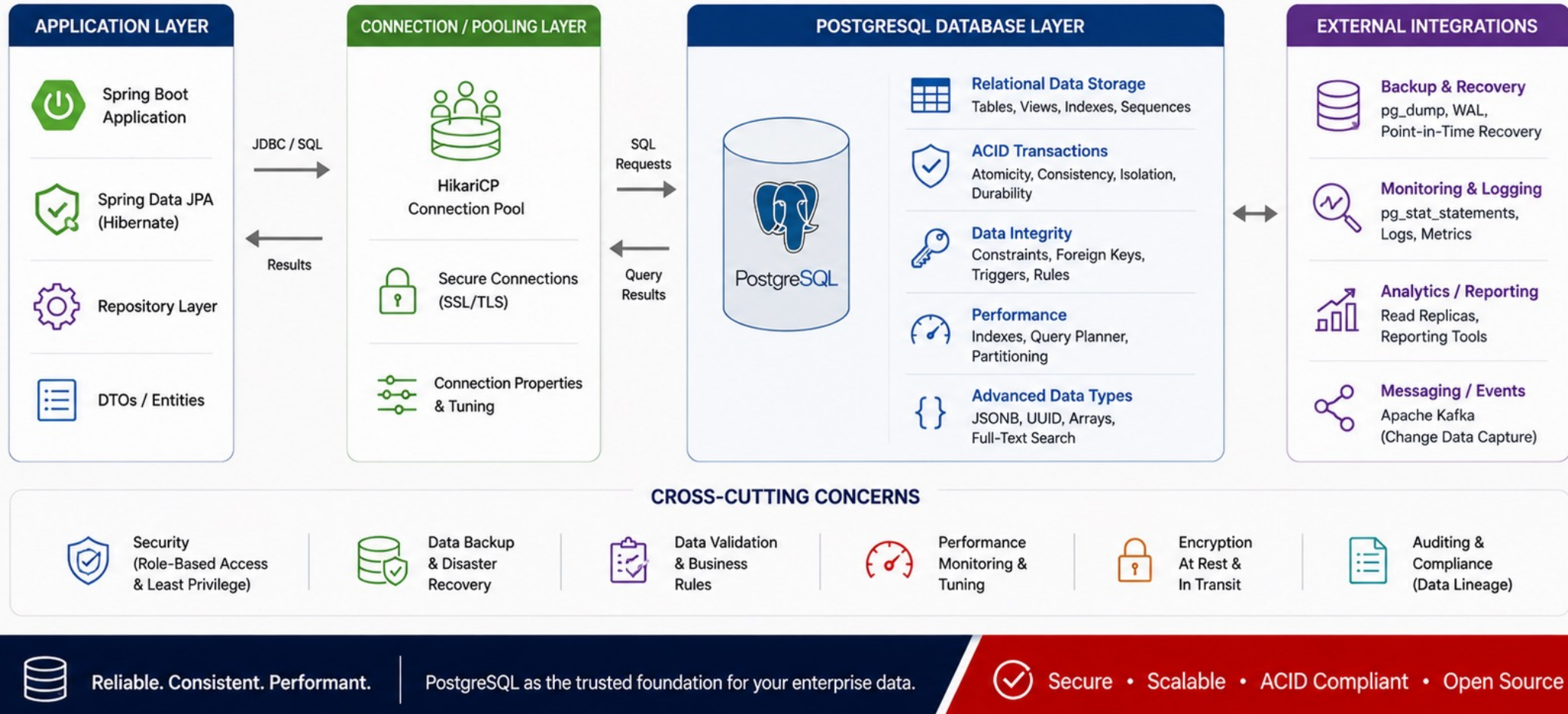
```
V1__create_customer_table.sql    V2__create_interaction_table.sql  
V3__index_interaction_customer_occurred_at.sql
```

KEY TAKEAWAY A missing migration or an accidental Oracle reference is a Module 49 blocker -- PostgreSQL and Flyway are the only path from day one.



POSTGRESQL PERSISTENCE ARCHITECTURE

A robust, scalable and secure data persistence architecture using PostgreSQL as the system of record for enterprise applications.



KAFKA MESSAGING ARCHITECTURE

CustomerInteractionRecordedV1 is the versioned event contract Module 49 publishes and later modules consume.

EVENT FIELDS

eventId, eventType, eventVersion.
occurredAt, correlationId, actor.
customerId, interactionId, channel.
Topic: crm.customer.interactions.v1.

DESIGN RULES

- Partition key = customer id, for ordered per-customer delivery.
- Publish only after the documented consistency strategy (ADR).
- Additive fields are safe; breaking changes bump the version.
- Consumers must dedupe on eventId -- delivery is at-least-once.

CustomerInteractionRecordedV1 (record)

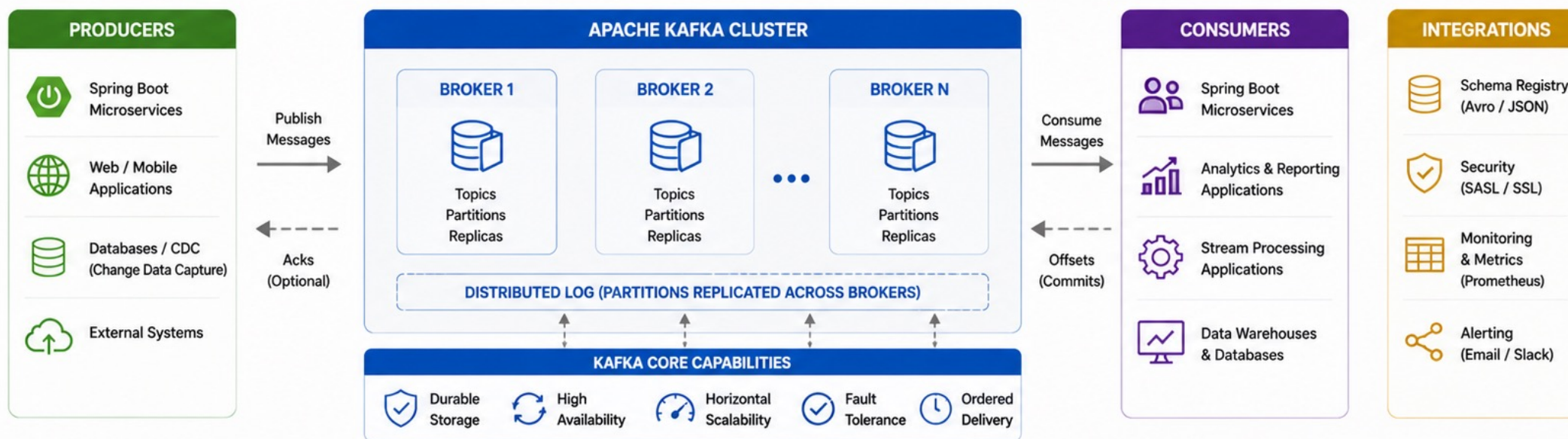
```
record CustomerInteractionRecordedV1(UUID eventId, String eventType, int eventVersion,  
    Instant occurredAt, String correlationId, String actor, UUID customerId, UUID interactionId, String channel) {}
```

KEY TAKEAWAY One event for one successful create -- the payload must include lab-request-001 and Amina's customer id, every time.



KAFKA MESSAGING ARCHITECTURE

A scalable, durable and fault-tolerant event streaming platform for real-time data integration and asynchronous communication.



DATA FLOW

- 1** Producer publishes messages to a topic
- 2** Kafka stores messages in partitions across brokers
- 3** Consumers read messages at their own pace
- 4** Offsets are committed to keep track of consumption
- 5** Messages are processed and used by downstream systems

KEY BENEFITS

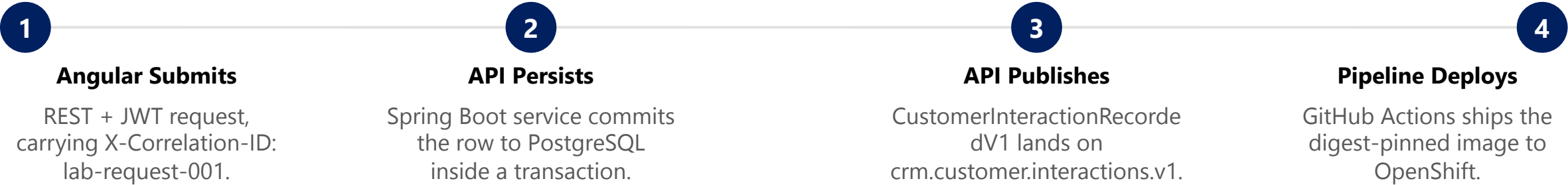
- Decoupled Architecture
- Scalable & Elastic
- Real-time Data Streaming
- Reliable & Durable (Replicated)
- Supports Multiple Consumers
- Enables Event-driven Architecture

FULL-STACK INTEGRATION ARCHITECTURE

Every arrow between Angular and OpenShift is an integration point -- and a place contract drift can hide.

Integration Point	Protocol	Contract Discipline
Angular UI <-> Spring Boot API	REST + JWT	Endpoint shape and Problem Details errors, drafted now.
Spring Boot API <-> PostgreSQL	Spring Data JPA	Flyway migrations own schema -- never auto-DDL.
Spring Boot API <-> Kafka	Async publish/consume	Versioned event, keyed by customer id.
GitHub Actions <-> OpenShift	oc apply / rollout status	Digest-pinned image, gated deploy.

END-TO-END TRACE



KEY TAKEAWAY One request traced from Angular through PostgreSQL and Kafka to a live OpenShift pod is stronger evidence than five confident claims.

TRY IT YOURSELF — EXERCISE 1: FREEZE THE CAPSTONE STACK

Reinforces: the Angular → REST → Spring Boot → PostgreSQL → Kafka stack you just walked, committed to writing before any capstone code exists.

STEPS (notes/lab48-stack-freeze.md)

Step	What To Do
1 — Components	List the five layers you just saw: Angular SPA, REST API, Spring Boot services, PostgreSQL, Kafka topics.
2 — Two ADRs	Write two short ADRs with a decision, a reason, and one rejected alternative each.
3 — Non-Goals	Name what the capstone will NOT use as the taught path: Bitbucket, Oracle, React, SOAP.
4 — Scope	Pre-lab only — no repository, no code, no diagrams rendered yet.

ADR shape (two sentences each)

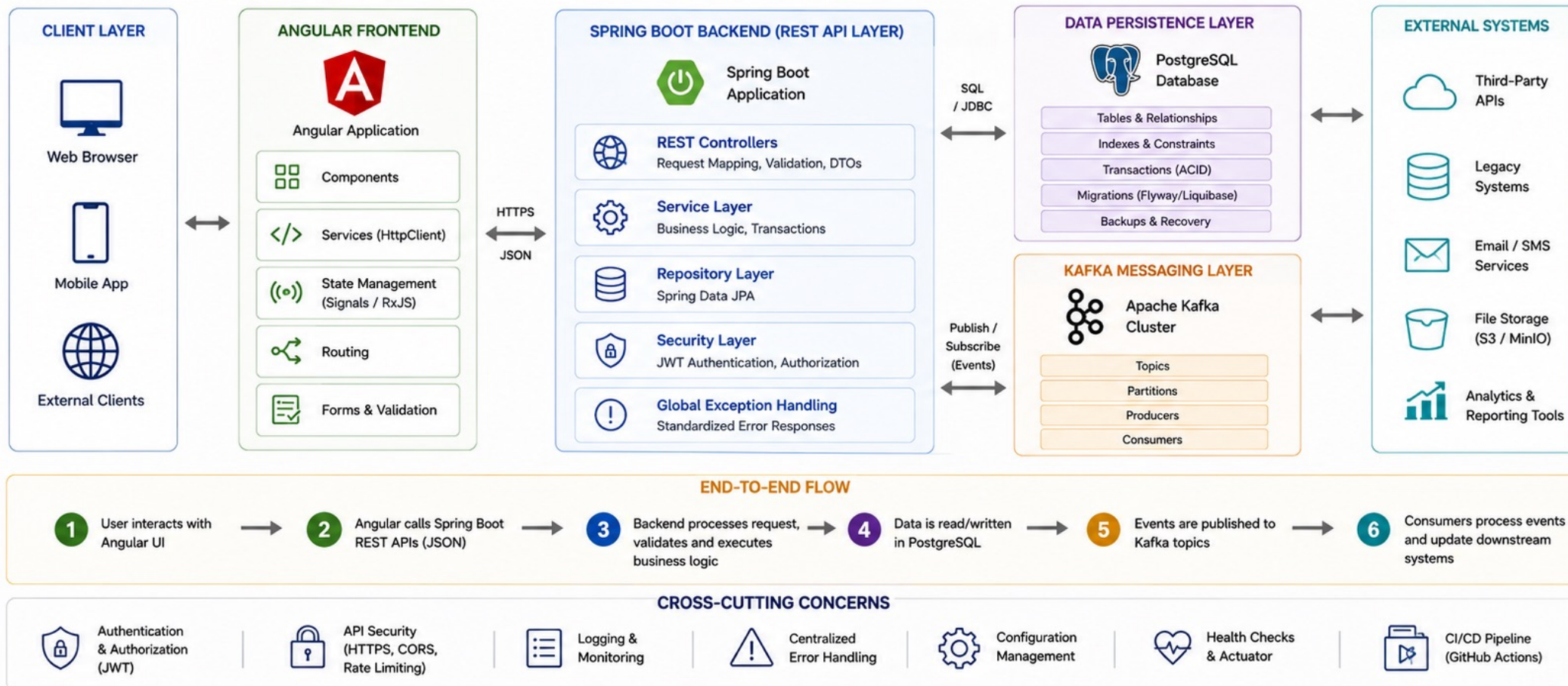
```
ADR-1  PostgreSQL as system of record    -> rejected: Oracle
ADR-2  GitHub Actions as CI/CD           -> rejected: Bitbucket Pipelines
Non-goals: React SPA, SOAP endpoints, Oracle-specific SQL
```

TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-stack-freeze.md (workspace: examples/module-48-exercises).



FULL-STACK INTEGRATION ARCHITECTURE

An enterprise-grade full-stack architecture showing end-to-end integration between Angular frontend, Spring Boot REST APIs, PostgreSQL, and Kafka.



Scalable • Secure • Resilient • Maintainable • Event-Driven



Better Integration. Better Performance. Better Experience.

GITHUB REPOSITORY STRATEGY

One repository, a disciplined branch model, and a docs tree a peer can reproduce the plan from alone.



Repo Layout

customer-management-
platform/: backend/, frontend/,
k8s/, docs/, .github/workflows/.



Branching Model

main is protected; feature
branches merge only through a
reviewed pull request.



Pull Request Discipline

Every PR links a backlog item or
ADR and passes CI before it can
merge.



Docs Tree

docs/architecture, docs/adrs,
and docs/nfrs live in the repo,
not in chat history.

KEY TAKEAWAY A peer must be able to clone the repo, read the docs tree, and reproduce the plan -- no verbal-only context allowed.



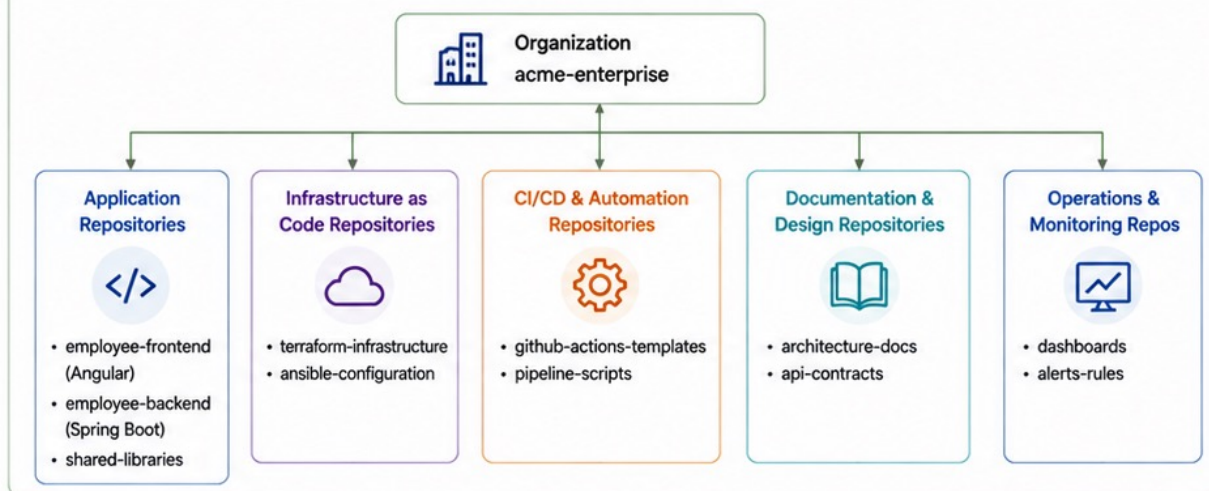
GITHUB REPOSITORY STRATEGY

A structured, secure and scalable repository strategy to manage the full-stack application source code, infrastructure, and automation.

STRATEGY PRINCIPLES

-  **Separation of Concerns**
Isolate application, infra and automation code.
-  **Security by Design**
Least privilege access and protected branches.
-  **Consistency**
Standardized structure, naming, and workflows.
-  **Traceability**
Clear versioning, tagging and release management.
-  **Scalability**
Modular repositories allow teams to scale independently.

REPOSITORY STRUCTURE



COLLABORATION WORKFLOW



BRANCH STRATEGY

-  **main**
Production-ready code. Protected branch.
-  **develop**
Integration branch for ongoing development.
-  **feature/***
Feature development branches.
-  **release/***
Release preparation branches.
-  **hotfix/***
Critical fixes for production.

NAMING CONVENTION

- Application Repos : <app-name>-<layer>
- Infra Repos : <purpose>-infra
- Automation Repos : <purpose>-automation
- Docs Repos : <purpose>-docs
- Ops Repos : <purpose>-ops

BEST PRACTICES

-  Protect main branch
-  Require PR & Code Review
-  Enforce Status Checks
-  Use Semantic Versioning
-  Keep README & Docs Updated
-  Manage Access with Teams
-  Automate with GitHub Actions

GITHUB ACTIONS CI/CD ARCHITECTURE

The pipeline gate order is fixed here -- never deploy an artifact that has not been verified, scanned, and published first.

Stage	Purpose
build	Compile Angular and Spring Boot; produce build artifacts.
verify	Run the full test suite -- backend and Angular, both must be green.
scan	Secret, dependency, and image scans -- criticals block unless exceptioned.
publish / deploy (gated)	Push the digest-identified image; roll out to OpenShift only after every prior stage passes.

PIPELINE HABITS

**Secrets Stay Scoped**
Registry and OpenShift credentials live in Actions secrets, never in YAML.

**Identity Flows Forward**
The same verified tag/digest moves stage to stage -- no silent rebuild.

**Approvals Where Required**
A deploy gate can require an explicit human approval, per GitHub Environments policy.

**Reports Preserved**
Test and scan reports are saved as pipeline artifacts, not discarded.

KEY TAKEAWAY A pipeline that deploys before its scan stage completes is not gated -- it is just automated, which is a very different guarantee.



GitHub Actions CI/CD Architecture

Automate, Build, Test, Secure, and Deploy – End to End



Consistent Pipelines. Secure Deliveries. Reliable Deployments.

TRY IT YOURSELF — EXERCISE 2: SKETCH THE ACTIONS PIPELINE

Reinforces: the GitHub repository and Actions CI/CD architecture you just saw, turned into a per-event job table for a fullstack capstone.

STEPS (notes/lab48-actions-plan.md)

Step	What To Do
1 — Event Table	Fill three rows: pull_request, push to main, tag v* — and the jobs each one runs.
2 — Fullstack Jobs	Name both halves: Angular npm ci / ng test / ng build, and Maven verify.
3 — Gates	Add the quality gates: dependency scan, SAST, container build and scan.
4 — Deploy Ownership	State who may deploy to OpenShift and that it stays manual-approval only.

Event → jobs policy

```
pull_request : ng test + mvn verify + scans      (no deploy)
push main    : verify + package once + image build (no deploy)
tag v*       : verify + package + manual OpenShift deploy
```

TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-actions-plan-sketch.md (workspace: examples/module-48-exercises).

INFRASTRUCTURE AUTOMATION WITH TERRAFORM

Terraform makes the OpenShift project's shape reviewable and repeatable -- not a set of manual clicks nobody wrote down.

TERRAFORM REVIEW POINTS

Declarative resources describe the desired OpenShift project state.
terraform plan is reviewed before terraform apply ever runs.
State is stored and locked appropriately -- never edited by hand.
Variables carry environment differences -- never hardcoded secrets.

WHY IT MATTERS FOR RELEASE

- A project or quota drift can silently break an OpenShift deploy.
- IaC changes go through the same PR review as application code.
- Reproducible infrastructure is what makes Module 52's evidence trustworthy.
- This module plans the IaC strategy -- Module 51 executes it as a pipeline stage.

Plan-before-apply discipline (concept)

```
terraform plan -out=tfplan    # reviewed by a teammate  
terraform apply tfplan       # only the reviewed plan is applied
```

KEY TAKEAWAY An unreviewed terraform apply run directly against the shared OpenShift cluster is exactly the undocumented change this discipline prevents.

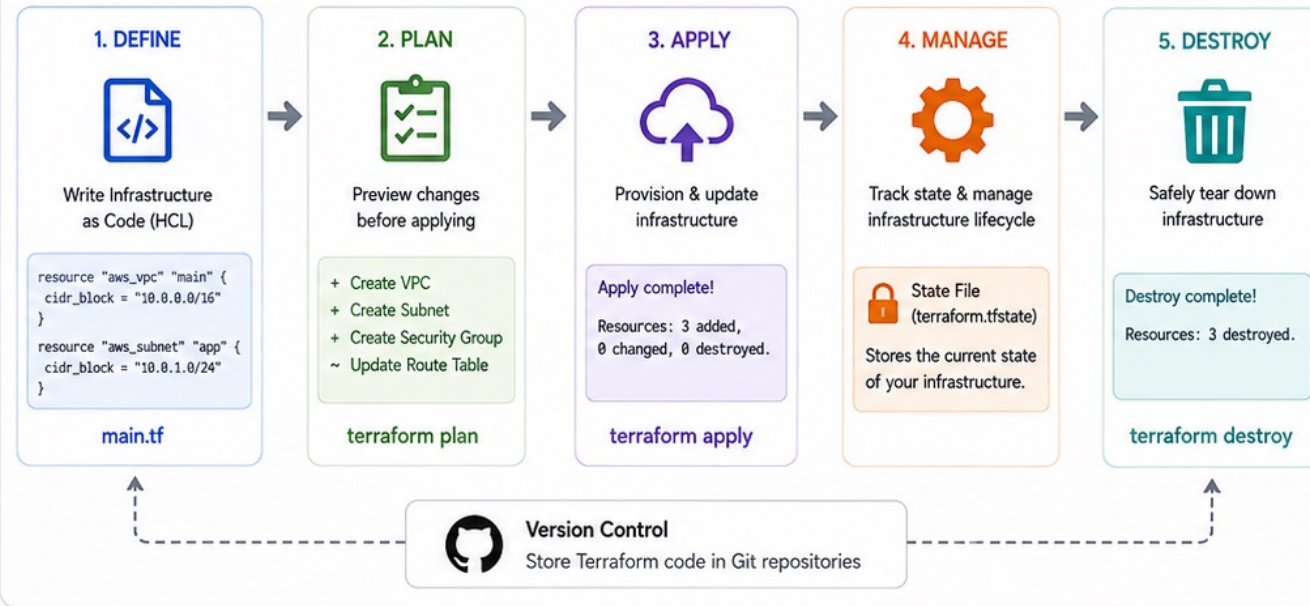
INFRASTRUCTURE AUTOMATION WITH TERRAFORM

Define, Provision, and Manage Infrastructure as Code

KEY BENEFITS

-  **Infrastructure as Code**
Version-controlled, repeatable and consistent.
-  **Automation & Speed**
Provision infrastructure in minutes, not days.
-  **Consistency & Compliance**
Ensure standards and policies across environments.
-  **Cost Efficiency**
Track, optimize and reduce infrastructure costs.
-  **Scalability**
Easily scale resources to meet demand.

TERRAFORM AUTOMATION WORKFLOW



SUPPORTED PROVIDERS

-  Amazon Web Services
-  Microsoft Azure
-  Google Cloud Platform
-  Red Hat OpenShift
-  HashiCorp Terraform Cloud
-  On-Premises / Private Cloud

EXAMPLE RESOURCES MANAGED

-  VPC / Networking
-  Compute Instances
-  Databases
-  Storage Buckets
-  Security Groups
-  Load Balancers
-  Route Tables
-  IAM Roles & Policies
-  Auto Scaling
-  More Resources



Terraform enables consistent, repeatable, and scalable infrastructure provisioning across environments using Infrastructure as Code.

CONFIGURATION AUTOMATION WITH ANSIBLE

Configuration management keeps OpenShift nodes and shared tooling consistent -- planned here, executed as a pipeline stage in Module 51.

ANSIBLE REVIEW POINTS

Playbooks are idempotent -- running twice produces the same end state.
Inventory separates environments; no shared secrets across them.
Vault (or an equivalent secret manager) protects sensitive variables.
Configuration drift gets corrected by re-running the playbook, not a manual fix.

WHY IT MATTERS FOR RELEASE

- A manually patched node is undocumented and unreproducible.
- Consistent baseline configuration reduces surprising OpenShift deploy failures.
- Playbook changes are reviewed, matching the Terraform review discipline.
- This module plans the pattern -- Module 51 runs it as a gated stage.

Idempotency check (concept)

```
ansible-playbook site.yml    # first run: makes changes
ansible-playbook site.yml    # second run: reports zero changes
```

KEY TAKEAWAY A node fixed manually during an incident, with no playbook update to match, drifts from the documented baseline the moment anyone forgets exactly what they did.

CONFIGURATION AUTOMATION WITH ANSIBLE

Automate Configuration, Ensure Consistency, Accelerate Deliveries

KEY BENEFITS



Consistency
Ensure uniform configuration across environments.



Speed & Efficiency
Automate repetitive tasks and save time.



Reliability
Reduce human errors and configuration drift.



Infrastructure as Code
Version-controlled and repeatable automation.



Scalability
Manage from a few to thousands of servers.

ANSIBLE AUTOMATION WORKFLOW

1. INVENTORY



Define your managed hosts and groups

```
[webservers]
web1.example.com
web2.example.com

[databases]
db1.example.com

[all:vars]
ansible_user=ec2-user
```

2. PLAYBOOKS



Write automation in YAML (playbooks)

```
---
- hosts: webservers
  become: yes
  tasks:
    - name: Install Nginx
      yum:
        name: nginx
        state: present
    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

3. MODULES



Ansible executes modules on target hosts

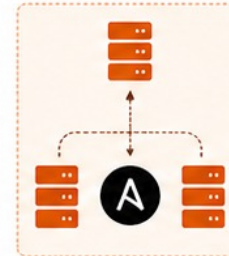
Common Modules

- package
- service
- copy
- template
- user
- file
- command
- yum/apt

4. EXECUTION



Ansible connects via SSH and executes tasks



5. RESULTS



Get real-time results and reports

PLAY RECAP

```
web1.example.com : ok=7
changed=2  unreachable=0
failed=0

web2.example.com : ok=7
changed=2  unreachable=0
failed=0

db1.example.com : ok=5
changed=1  unreachable=0
failed=0
```

HOW ANSIBLE WORKS



Control Node
(Ansible Installed)



Inventory
(Hosts & Groups)



Playbook
(YAML)



Modules
(Execute Tasks)



Managed Nodes
(Target Servers)



Results & Reports
(Real-Time Feedback)

COMMON AUTOMATION USE CASES



Application
Deployment



User Management



Security Hardening



Configuration
Management



Database
Setup



Service Management



Cloud & Hybrid
Automation

BEST PRACTICES

- ✓ Use variables and templates for reusability
- ✓ Organize playbooks with roles
- ✓ Keep playbooks idempotent
- ✓ Store code in Git for version control
- ✓ Test in non-production before rollout

INTEGRATION



Git
(Version Control)



Jenkins / GitHub Actions (CI/CD Pipelines)



Terraform
(Infrastructure Provisioning)



Monitoring Tools
(Prometheus, Grafana)

ANSIBLE COMPONENTS



Ansible Control Node
Where Ansible is installed and commands are run.



Managed Nodes
Servers, VMs, containers managed by Ansible.



Playbooks
YAML files that define configuration and tasks.



Roles
Reusable collections of playbooks, tasks and files.



Ansible Galaxy
Repository for roles and collections.



Ansible automates configuration management, application deployment, and IT operations at scale.

TRY IT YOURSELF — EXERCISE 3: SCOPE TERRAFORM AND ANSIBLE

Reinforces: the Terraform provisioning and Ansible configuration split you just saw, bounded so the capstone never hand-edits infrastructure.

STEPS (notes/lab48-iac-plan.md)

Step	What To Do
1 — Split the Work	Write one line each: what Terraform provisions, what Ansible configures.
2 — State	Name where remote state lives and why it is never committed to Git.
3 — Idempotence	Give one Ansible task that is safe to run twice, and say how you know.
4 — Secrets Rule	State the rule: no secrets in HCL, YAML, or Git — injected at run time only.

Ownership boundary

```
terraform : OpenShift project, quotas, routes, managed Postgres
ansible   : app config, env vars, DB users, scheduled jobs
state     : remote backend, locked, never in Git; secrets injected
```

TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-iac-plan-sketch.md (workspace: examples/module-48-exercises).

OPENSIFT DEPLOYMENT ARCHITECTURE

OpenShift is this course's real, only deployment target -- not a side comparison to a training cluster.

OpenShift Object	Purpose
Project	The namespace boundary for the Northstar CRM's OpenShift resources.
Deployment / DeploymentConfig	Manages replica pods and rollout history for crm-api and the Angular UI.
Service	Gives each workload a stable, cluster-internal network identity.
Route	Exposes the Angular UI and API beyond the cluster over HTTPS.

WHY OPENSIFT HERE



Real Target, Not Comparison

Every deploy, smoke test, and rollback in Module 51 runs against OpenShift.



Routes Replace Ingress

OpenShift Routes are the first-class way this course exposes services externally.



Security Context Constraints

SCCs enforce pod security defaults platform-wide, not per-manifest.



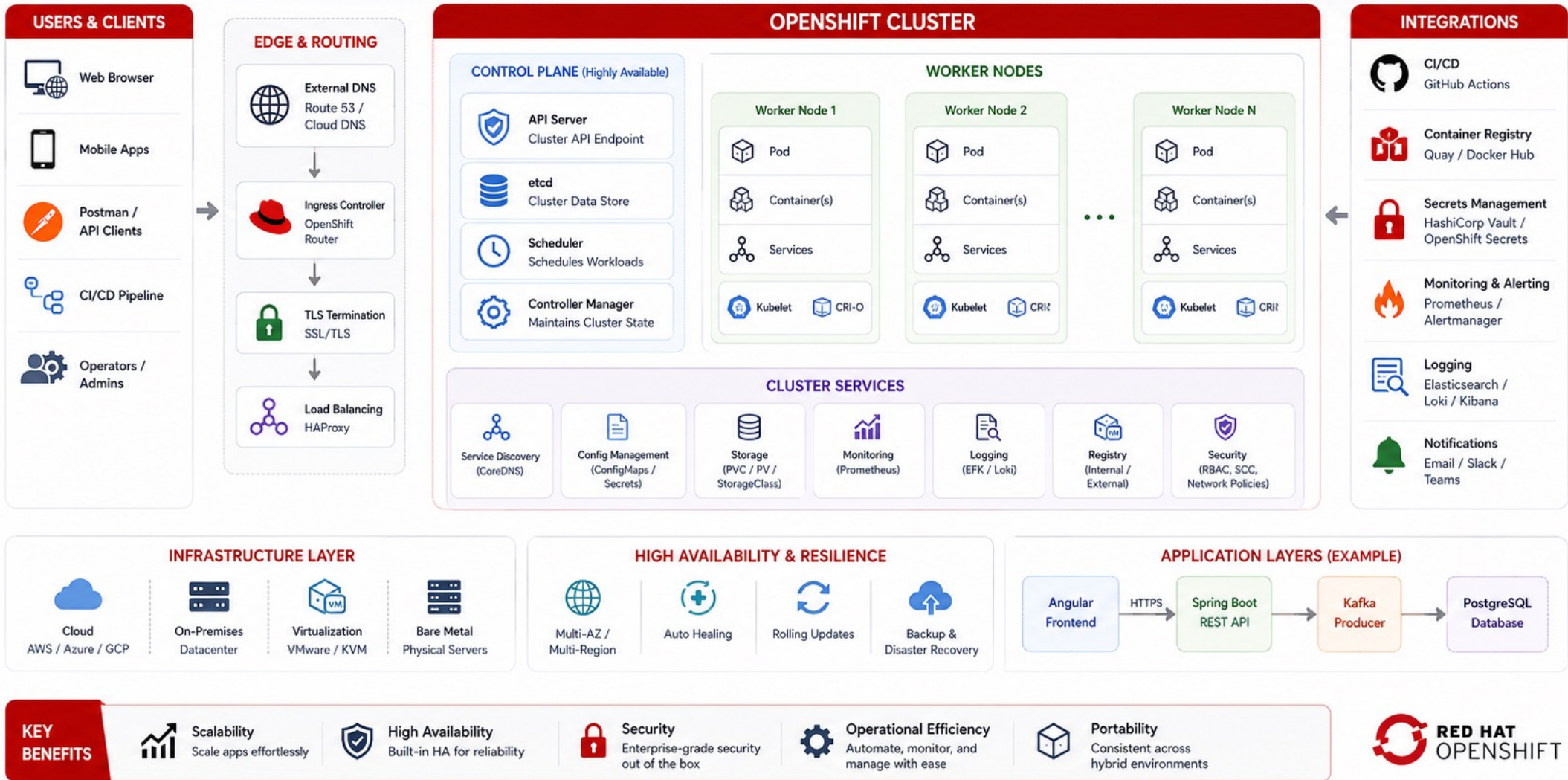
Built-In Image Policy

ImageStreams can track and trigger on new digests without extra tooling.

KEY TAKEAWAY State clearly: this capstone deploys to OpenShift, full stop -- every OpenShift object named here becomes real in Module 51.

OPENSIFT DEPLOYMENT ARCHITECTURE

Scalable, Secure, and Highly Available Enterprise Application Platform



AI USAGE PLAN FOR CAPSTONE

AI assistance is welcome across the capstone -- scoped to bounded, reviewed uses, never unattended decisions.



Scaffolding & Boilerplate

Angular component/service skeletons, Terraform modules, and Ansible playbook shells.



Code Explanation

Understanding an unfamiliar .tf file, GitHub Actions workflow, or legacy Angular pattern.



Troubleshooting & Docs

Explaining a cryptic pipeline or OpenShift error, and drafting a README from working code.



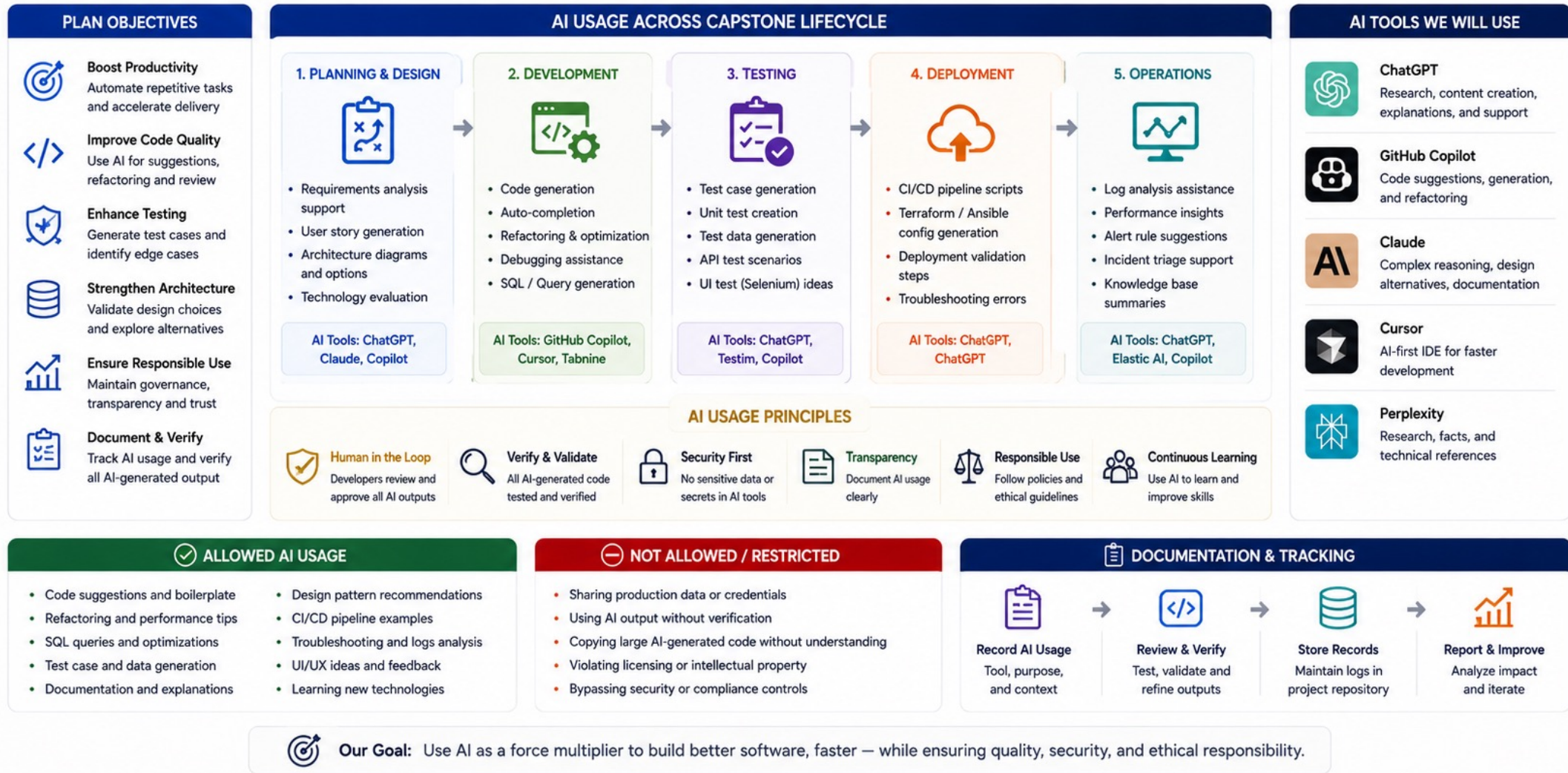
Policy-as-Code Drafting

Guardrail checks -- e.g. rules forbidding a public route or a floating image tag.

KEY TAKEAWAY AI speeds up scaffolding, explanation, and drafting -- it never gets the final word on what ships to OpenShift.

AI USAGE PLAN FOR CAPSTONE

Leverage AI Responsibly to Accelerate Development, Ensure Quality, and Drive Innovation



AI GOVERNANCE AND VERIFICATION PLAN

The Human Accountability Gate is what makes this capstone's AI usage safe -- every AI-assisted decision gets a bounded prompt and a review record.

HUMAN ACCOUNTABILITY GATE

Prompts are bounded and contract-first -- state assumptions, forbid secrets/public exposure.

Generated code gets reviewed critically, never rubber-stamped because it lints clean.

Nothing gets applied to OpenShift purely because an AI assistant suggested it.

Every review names at least one rejected or hardened suggestion, with a reason.

Bounded, contract-first prompt (example)

```
"Draft an OpenShift Route for crm-api in the staging Project. Require TLS edge termination, forbid a wildcard host, and do not include any real hostname or secret."
```

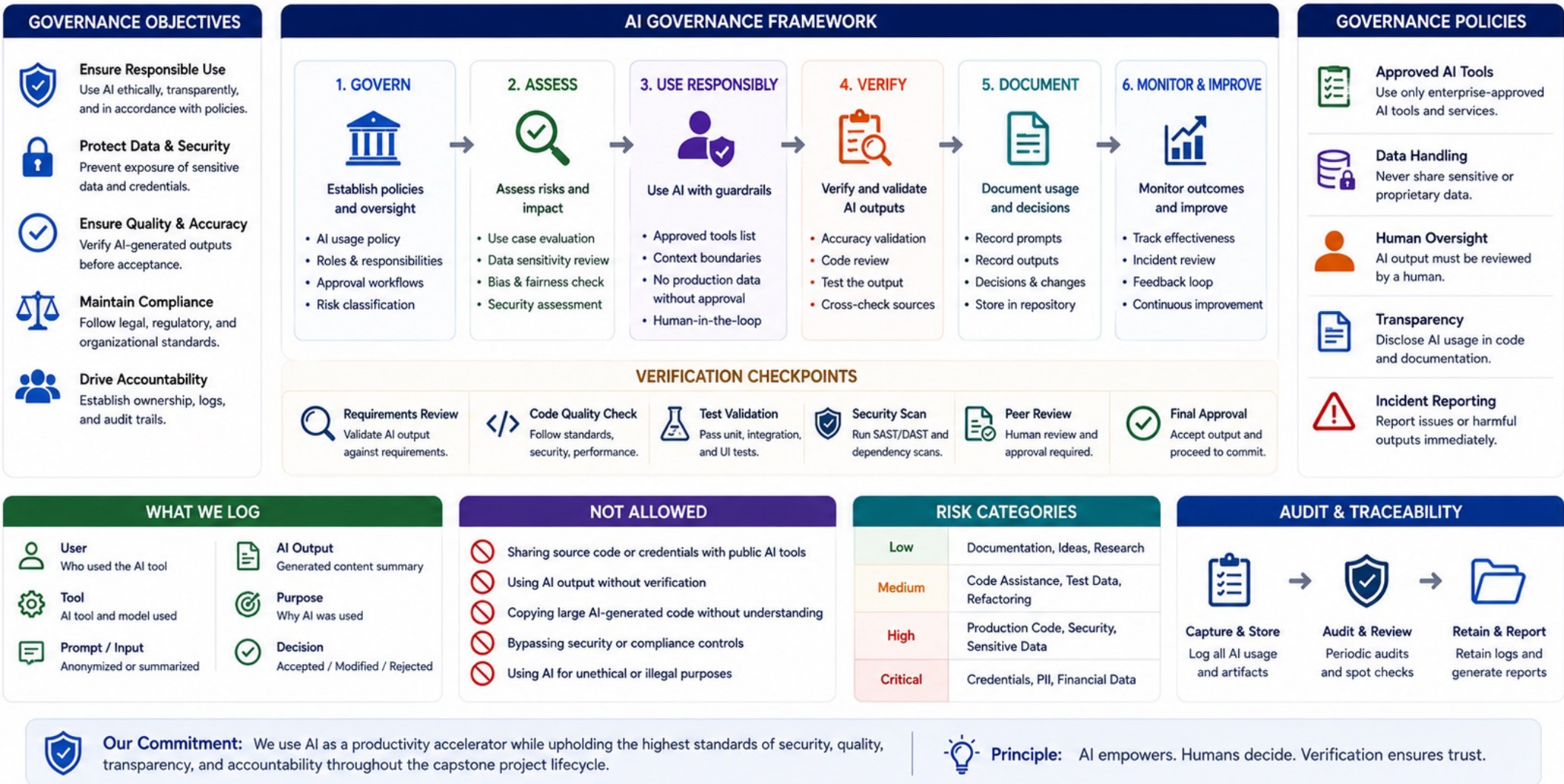
VERIFICATION DISCIPLINE

- Every AI-assisted change still passes the same GitHub Actions gates as any other change.
- A dated review record is written for each AI-assisted decision.
- 'The AI wrote it, so it must be right' is a named misconception to correct.
- AI usage disclosure belongs in the PR description, not left implicit.

KEY TAKEAWAY A human reads and reasons about every AI-assisted create/update/replace/destroy action before it reaches OpenShift -- no exceptions.

AI GOVERNANCE AND VERIFICATION PLAN

Ensure Responsible AI Usage, Verify Outputs, and Maintain Trust, Security, and Compliance



TRY IT YOURSELF — EXERCISE 4: DRAFT THE AI USAGE PLAN

Reinforces: the AI usage and governance slides you just covered, written as allowed / verify / forbidden rules the capstone team can actually apply.

STEPS (notes/lab48-ai-usage-plan.md)

Step	What To Do
1 — Allowed	List three tasks AI may draft: boilerplate, test drafts, HCL and YAML sketches.
2 — Verify	Write one verification check for Angular output and one for Flyway/SQL output.
3 — Forbidden	List what AI never receives or does: secrets, real customer data, production applies.
4 — Attribution	State how AI-assisted changes are disclosed in the pull request.

Three-column AI policy

```
allowed      : boilerplate, test drafts, HCL/YAML sketches, docs
verify       : ng test green + reviewed; Flyway runs forward AND back
forbidden    : secrets, real PII, prod applies, unreviewed merges
```

TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-ai-usage-plan.md (workspace: examples/module-48-exercises).

CAPSTONE ENVIRONMENT STRATEGY

Four environments, one promotion path -- local, CI, staging, and production OpenShift, in that order, never skipped.

Environment	Purpose
Local	Docker Compose runs Angular, Spring Boot, PostgreSQL, and Kafka on a laptop.
CI (GitHub Actions)	Ephemeral, ephemeral-database test run on every pull request.
Staging (OpenShift)	Pre-production verification -- smoke tests run here before promotion.
Production (OpenShift)	The approved, digest-pinned release -- reached only through a gated promotion.

ENVIRONMENT HABITS



One-Way Promotion

An artifact moves local -> CI -> staging -> production, never sideways or backward.



Config, Not Code, Changes

The same image and manifests are promoted -- only environment-scoped config differs.



GitHub Environments Enforce It

Protection rules gate who can promote to staging and production, detailed in Module 51.



Fixtures Stay Synthetic

Amina and Ravi's fixture data lives in every environment except production.

KEY TAKEAWAY Naming all four environments here means Module 51's GitHub Environments and approval-gate slides have a fixed target to configure.

TRY IT YOURSELF — EXERCISE 5: MAP THE ENVIRONMENT STRATEGY

Reinforces: the capstone environment strategy you just saw, mapped onto real OpenShift Projects with named promotion gates.

STEPS (notes/lab48-environment-strategy.md)

Step	What To Do
1 — Four Environments	Map dev, test, stage, and prod each to one OpenShift Project name.
2 — What Differs	Say what changes per environment: config and data, never the image.
3 — Promotion Gates	Name the evidence required to move from stage to prod.
4 — Data Rule	State that only synthetic fixtures such as CUS-1001 exist below prod.

Environment map

```
dev / test / stage / prod -> crm-dev, crm-test, crm-stage, crm-prod
same image digest promoted; only ConfigMap and Secret values differ
stage -> prod needs: green smoke + approval + rollback rehearsed
```

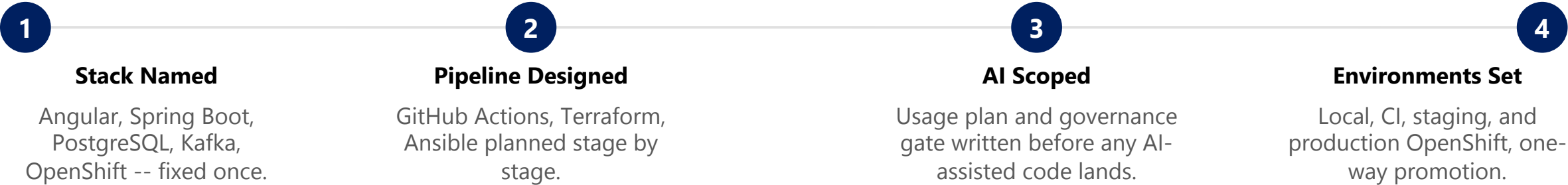
TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-environment-strategy.md (workspace: examples/module-48-exercises).

END-TO-END ARCHITECTURE REVIEW

Every decision this module made, traced once more from Angular to a live OpenShift release.

Stage	What Happens	Owned By
Plan	Stack, architecture, CI/CD, IaC, AI usage, and environments frozen.	Module 48 (this module)
Build	Angular UI and Spring Boot/Kafka backend implemented against the plan.	Modules 49-50
Release	GitHub Actions gates, Terraform/Ansible run, image ships to OpenShift.	Module 51
Defend	Live demo, Q&A, and retrospective against this module's frozen decisions.	Module 52

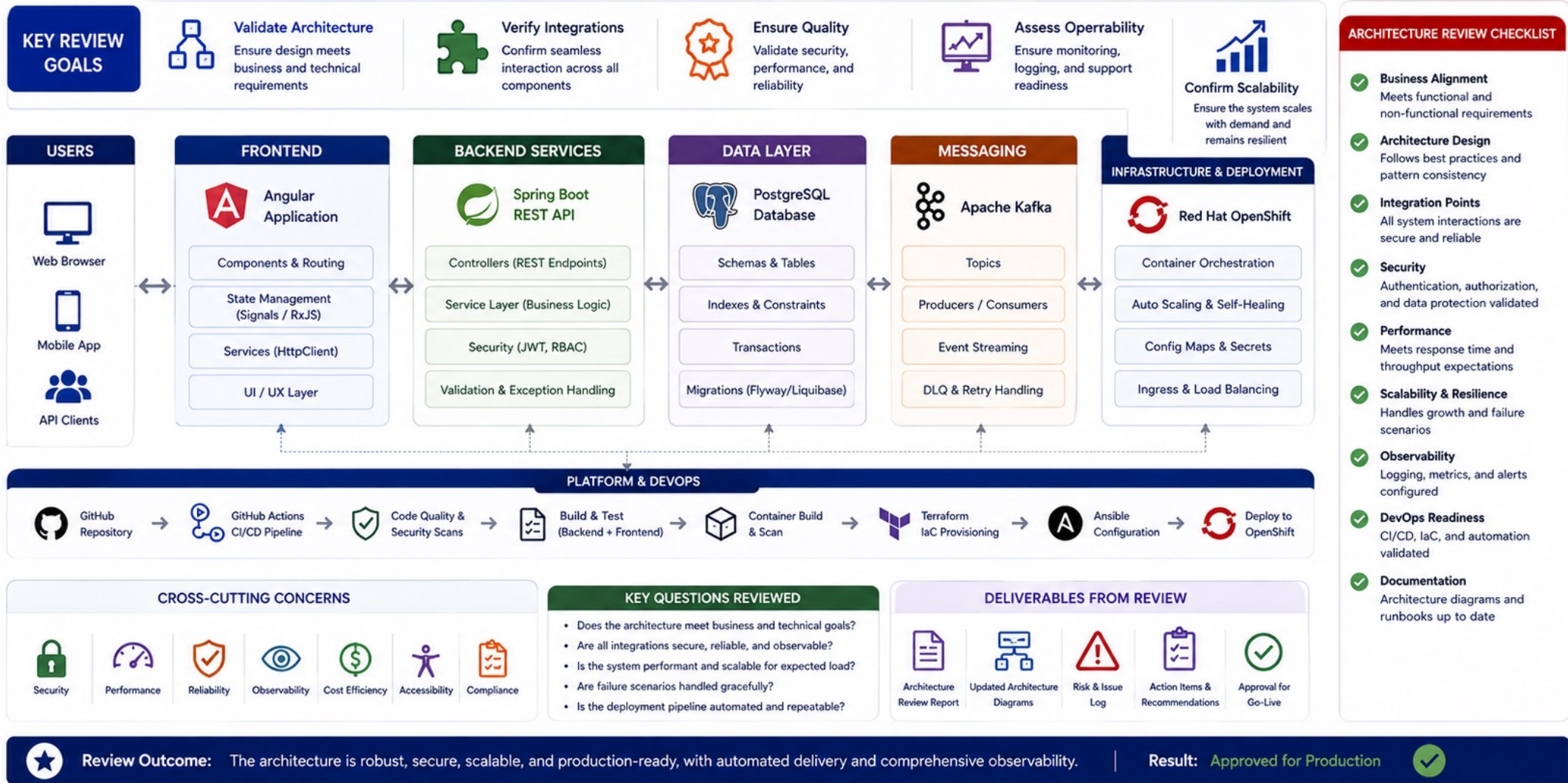
FULL PICTURE



KEY TAKEAWAY Nothing in Labs 49-51 or Module 51 should silently diverge from a decision frozen here -- especially OpenShift as the real deployment target.

END-TO-END ARCHITECTURE REVIEW

Holistic Review of Capstone Architecture, Integrations, and Operational Excellence



TRY IT YOURSELF — EXERCISE 6: SEED THE BACKLOG AND SELF-CHECK

Capstone pre-lab gate: seed the CAP-12 story Lab 49 builds from, then confirm all five earlier plans genuinely exist before opening the graded lab.

STEPS (notes/lab48-backlog-readiness.md)

Step	What To Do
1 — CAP-12 Seed	Write CAP-12 'Record customer interaction' with acceptance criteria and fixtures.
2 — Two More Stories	Add two CAP-style stories sized for one session each.
3 — Plan Roll-Call	Confirm the five earlier notes files exist and are filled in, not stubbed.
4 — Pass Mark	Mark Pass only if every earlier exercise is complete; otherwise go back.

Story shape

```
CAP-12  Record customer interaction for CUS-1001
  given an ACTIVE customer, when an interaction is recorded,
  then it persists and an InteractionRecorded event is published
```

TRY IT NOW — Complete Exercise 6 before Lab 48: exercises/exercise-06-backlog-and-readiness.md (workspace: examples/module-48-exercises).

LAB OVERVIEW — CAPSTONE ARCHITECTURE AND PLANNING

Stack diagram · AI plan · Actions plan · TF/Ansible plan · env strategy · backlog seeds /

BUSINESS SCENARIO

- Leadership freezes: No Lab 49+ merge without a reviewed architecture and delivery plan. Your plan must name Angular, REST, Spring Boot, PostgreSQL, Kafka, Docker, OpenShift, GitHub Actions, and Terraform/Ansible—not Bitbucket, Oracle, React, or SOAP.

LEARNING OBJECTIVES

- Document the mandatory fullstack data and control paths Plan GitHub Actions quality gates before writing workflow YAML in Lab 51 Bound Terraform/Ansible scope and secret handling Define environment promotion and OpenShift Project strategy Seed an acceptance-backed backlog for implementation labs

WHAT YOU BUILD

- docs/architecture.md + diagrams (Angular → REST → Spring Boot → PostgreSQL; Kafka; OpenShift); docs/ai-usage-plan.md (allowed uses, verification, forbidden); docs/github-actions-plan.md (PR/main/tag jobs, gates, artifacts); docs/terraform-ansible-plan.md (scope, state, idempotence, secrets)

KEY TAKEAWAY This Module 48 lab produces the ****capstone planning pack**** for Northstar CRM: architecture, AI governance, GitHub Actions, Terraform/Ansible, and environment strategy that Labs 49–51 implement against.

LAB TASKS AND DELIVERABLES

Capstone Architecture and Planning — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Open starter/README.md.
2	Copy starter/ into java-bootcamp/examples/lab48-capstone-plan.
3	Fill every TODO in docs templates (architecture, AI, CI/CD, IaC, environments).
4	Produce at least one Mermaid (or exported) diagram; evidence under notes/screenshots/lab-48/.
5	Mark timed-path Pass criteria. Continue remaining GUIDE steps as homework.
6	docs/architecture.md + diagrams (Angular → REST → Spring Boot → PostgreSQL; Kafka; OpenShift)
7	docs/ai-usage-plan.md (allowed uses, verification, forbidden)
8	docs/github-actions-plan.md (PR/main/tag jobs, gates, artifacts)
9	docs/terraform-ansible-plan.md (scope, state, idempotence, secrets)
10	docs/environment-strategy.md (dev/test/stage/prod mapping to OpenShift)
11	docs/backlog.md with CAP-style stories (at least CAP-12 seed for Lab 49)
12	No secrets, real customer data, or Bitbucket/Oracle/React/SOAP as taught path

EXPECTED DELIVERABLES

- docs/architecture.md + diagrams (Angular → REST → Spring Boot → PostgreSQL; Kafka; OpenShift) docs/ai-usage-plan.md (allowed uses, verification, forbidden) docs/github-actions-plan.md (PR/main/tag jobs, gates, artifacts) docs/terraform-ansible-plan.md (scope, state, idempotence, secrets) docs/environment-strategy.md (dev/test/stage/prod mapping to OpenShift) docs/backlog.md with CAP-style stories (at least CAP-12...

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs/Evidence 10

KEY TAKEAWAY Lab 48 is graded capstone work. Pre-lab exercises 1→6 must be Pass before the lab path; keep evidence under notes/screenshots and never commit secrets.

MODULE 48 SUMMARY

In this module, we froze the Northstar CRM's Angular, REST, PostgreSQL, Kafka, and OpenShift architecture and delivery plan.

KEY CONCEPTS COVERED



Stack Frozen

Angular, Spring Boot, PostgreSQL, Kafka, and OpenShift -- named once for Labs 49-51.



Full-Stack Integration

Angular -> REST+JWT -> Spring Boot -> JPA
-> PostgreSQL, plus Kafka, traced end to end.



CI/CD Designed

GitHub Actions gate order planned before
Module 51 builds the real workflow.



IaC Planned

Terraform and Ansible strategies set, ready
for Module 51's gated pipeline stages.



OpenShift Is Real

Projects, Deployments, Services, and Routes -
the capstone's deployment target.



AI Scoped and Governed

A usage plan plus a Human Accountability
Gate for every AI-assisted decision.

MODULE FLOW RECAP

1

1. Name Stack

2

2. Design Architecture

3

3. Plan Pipeline

4

4. Scope AI

5

5. Set Environments

KEY TAKEAWAY Every decision this module made -- stack, architecture, CI/CD, IaC, AI usage, and environments -- is what Labs 49-51 and Module 51 build against, unchanged.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of the capstone stack, the frontend/API/database architecture, and full-stack integration.

1. Which deployment platform is NOT used in this course's capstone?

- A. OpenShift
- B. Kubernetes
- C. k3s**
- D. Docker

3. What must own the PostgreSQL schema, per this module's rule?

- A. Hibernate auto-DDL
- B. Flyway numbered migrations**
- C. A manual SQL script run once
- D. The Angular service layer

2. What database does every environment in this capstone use?

- A. Oracle
- B. PostgreSQL**
- C. MongoDB
- D. MySQL

4. In the Angular frontend architecture, where should a JWT and correlation header be attached to a request?

- A. Inside each component's constructor
- B. In a shared HTTP interceptor**
- C. Manually on every HttpClient call
- D. In the Spring Boot controller

KEY TAKEAWAY No k3s in the capstone; PostgreSQL everywhere; Flyway owns the schema; interceptors attach JWT and correlation headers once, centrally.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on CI/CD, infrastructure automation, AI governance, and environment strategy.

5. In the GitHub Actions pipeline gate order, what must happen before deploy?

- A. Nothing -- deploy can run first for speed
- B. Build, verify, scan, and publish must all pass**
- C. Only a successful build is required
- D. Only the security scan is required

7. What is the correct one-way environment promotion path for this capstone?

- A. Production -> staging -> CI -> local
- B. Local -> CI -> staging -> production**
- C. CI -> production -> staging -> local
- D. Staging -> production, skipping CI

6. Per this module's AI governance rule, what must every AI-assisted review include?

- A. Nothing -- AI suggestions are accepted as-is
- B. At least one rejected or hardened suggestion, with a reason**
- C. A screenshot only
- D. Approval from the AI tool itself

8. What OpenShift object exposes the Angular UI and API beyond the cluster over HTTPS?

- A. A ConfigMap
- B. A Route**
- C. A Secret
- D. A PersistentVolumeClaim

KEY TAKEAWAY Deploy only after build/verify/scan/publish pass; every AI review names a rejected suggestion; promotion is one-way; a Route exposes services externally.

*"Freeze the plan before you freeze the code -
- Angular, REST, Kafka, and a real OpenShift
target everyone can defend."*

MODULE 48 COMPLETE

Capstone Planning and Architecture

You can now name the Northstar CRM's Angular, REST, Kafka, PostgreSQL, and OpenShift architecture; design a GitHub Actions, Terraform, and Ansible delivery pipeline; and plan AI usage, governance, and environments the rest of Week 6 builds against.